

AWS GenAI 課程 · 第 10 單元

資料庫 RDS、Aurora 與快取

盧建霖 • 2026-10-01

機器掛了可以重開，
資料掉了就真的沒了。

Agenda

- 01 RDS：受管與兩種多一台
- 02 Aurora：運算跟儲存分開
- 03 快取：先問記憶體
- 04 懂意思的資料庫與切換實測

TODAY'S GOAL

替專題的資料庫做對三個選擇：RDS 或 Aurora、要不要多一台、前面放不放快取；再親手切一次量出斷幾秒。

前兩段講資料怎麼不掉、服務怎麼不倒，第三段講怎麼讓資料庫少做事，最後接到 AI 應用與實作。

先定義名詞，後面才好講

這六個詞今天會反覆出現；Aurora、語意快取到那一段再定義

關聯式資料庫 **relational database**

資料放在像試算表的表格裡，每列一筆、每欄一個欄位，用 SQL 查。

RDS Relational Database Service

AWS 代管的關聯式資料庫服務，MySQL、PostgreSQL 都在裡面。

多可用區部署 **Multi-AZ**

在另一個 AZ 放一台同步的備援，平常不能用，主要的掛了才接手。

讀取複本 **Read Replica**

非同步抄一份資料，平常就能拿來讀。

故障切換 **failover**

主要的掛了，RDS 把資料庫名稱改指到備援，讓它變成新的主要。

快取 **cache**

把常被問的答案放在記憶體裡，下次直接拿。

資料庫交給 **AWS** 養，再多一台

- 一、受管買的是修補、備份、切換有人保證會做
- 二、**Multi-AZ** 是保險，**standby** 平常不能讀
- 三、**Read Replica** 是分身，非同步、不會自動接手

同一個 MySQL，責任線換了位置

放在 EC2 上，從作業系統到故障切換全是你的事；放在 RDS 上，這些變成 AWS 的事

工作	EC2 自建	RDS 受管
實體機房、硬體	AWS	AWS
作業系統安裝與修補	你	AWS
資料庫軟體安裝與修補	你	AWS
備份與還原	你	AWS (自動)
故障切換到另一個 AZ	你	AWS (勾 Multi-AZ)
資料表設計、查詢、帳號權限	你	你

紅色是你要做的事；最後一列兩邊都是你，受管服務管不到你的資料表跟權限

- 1 左欄六件工作，紅色是你要做的、綠色是 AWS 做的
- 2 換成 RDS，作業系統、資料庫軟體、備份、切換都轉成 AWS
- 3 最後一列兩邊都是你：資料表設計、查詢、帳號權限

裝 MySQL 不難，難的是每一天

天真做法：開一台 EC2、裝 MySQL、應用程式連上去，第一個月都很好

時間	發生什麼事	自建的你要做什麼
第 2 個月	MySQL 出了安全修補	挑半夜停機、升級、祈禱沒壞
第 4 個月	工程師手滑刪了一張表	翻備份，發現腳本上個月就壞了
第 7 個月	那台機器所在的 AZ 出事	別的 AZ 開新機器、還原、改連線
每一天	磁碟快滿了嗎？慢查詢是誰？	自己寫監控、自己看

每一件都不難，難的是每一件都要有人一直做，而且做錯的那次剛好就是出事那次。

RDS 替你做掉三件事

快照 (snapshot)：資料庫在那一刻的完整複本；保留期主控台建立預設 7 天、CLI 預設 1 天

工作	RDS 怎麼做	你要做的
修補	排在你指定的每週維護時段套用	選離峰時段，看一眼待套用清單
備份	每天一次快照，加交易紀錄	決定保留幾天，最長 35 天
故障切換	勾 Multi-AZ，主要掛了自動切	勾那一格

標成強制的更新，過了期限會在維護時段自動套。講師補充：同規格 RDS 通常比自建貴。

手滑刪表：還原到刪表前一分鐘

時間點還原（point-in-time recovery）：把資料庫還原到保留期內過去某一刻的樣子

1 交易紀錄每 5 分鐘上傳到 S3

最新還原點通常是幾分鐘前，以 latest restorable time 為準。

2 拿那一刻之前的快照，再重播紀錄

這是講師的白話說明；交易紀錄是每一筆改動的流水帳。

3 還原出來的是一個新的資料庫

原本那個不動，應用程式要改連過去，或把資料搬回來。

要碰機器本身，才回 **EC2** 自己養

引擎 (engine)：實際存資料、跑查詢的軟體；RDS 有 MySQL、PostgreSQL、MariaDB、Oracle、SQL Server、Db2，外加 Aurora

情況	為什麼 RDS 不行
要登入作業系統、裝清單外的外掛	不讓你登入底下的機器（RDS Custom 例外）
用的引擎或版本 RDS 不支援	換版本或引擎，或回 EC2 自己裝
軟體授權按實體機器算	RDS 不讓你挑機器（講師補充）

RDS 給你的是資料庫；需要碰到機器的時候，才回去用 **EC2**。

交給 RDS，安全就是 AWS 的事？

× 大家通常會想

受管了，資料庫的安全都歸 AWS 管。

受管只是把責任線往上推到作業系統與資料庫軟體。

✓ 實際上

誰能連、密碼放哪、資料表、加密，永遠是你的。

講師經驗：帳號權限開太大，比 AZ 掛掉常見得多。

QUIZ 01 · CLF-C02

一家公司希望在 **AWS Cloud**（雲端）中執行關聯式資料庫。公司希望使用一項受管服務，能夠自動安裝資料庫並定期執行軟體更新。哪一項 **AWS** 服務可以滿足這些需求？

A

Amazon S3

B

Amazon RDS

C

Amazon Elastic Block Store (Amazon EBS)

D

Amazon DynamoDB

想 30 秒，等一下翻下一頁對答案。

QUIZ 01 · CLF-C02 · ANSWER

一家公司希望在 **AWS Cloud**（雲端）中執行關聯式資料庫。公司希望使用一項受管服務，能夠自動安裝資料庫並定期執行軟體更新。哪一項 **AWS** 服務可以滿足這些需求？

A

Amazon S3

B

Amazon RDS

C

Amazon Elastic Block Store (Amazon EBS)

D

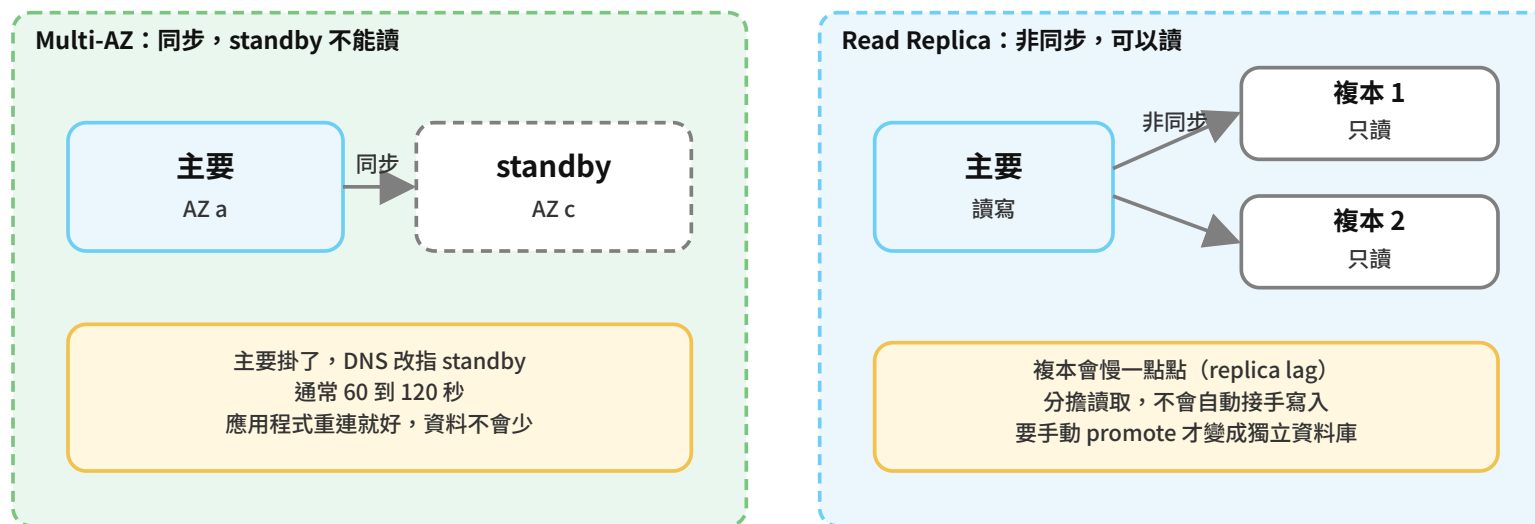
Amazon DynamoDB

B。「關聯式、受管、自動安裝並定期更新軟體」是 RDS；EBS 是磁碟不是資料庫，DynamoDB 是 NoSQL，S3 是物件儲存。

兩台長得很像：一個保命，一個分擔

都是主要資料庫旁邊多一台、資料都會抄過去；要解的問題完全不同

Multi-AZ 保命，Read Replica 分擔讀取



- 1 左邊 Multi-AZ：同步抄給 standby，standby 不能讀
- 2 右邊 Read Replica：非同步抄給複本，複本可以讀
- 3 下方黃框：主要掛了，兩邊各自會怎樣

關鍵字只有兩個：同步、非同步

差別在主要資料庫什麼時候跟應用程式說「寫好了」

1 同步 (synchronous)：等備援也寫好才回

收到「寫好了」，備援一定也有，切換不會弄丟。

2 代價：每次寫入要多等一下

Multi-AZ 走這一種。

3 非同步 (asynchronous)：主要寫好就先回

再慢慢抄給複本；落後的那段叫 replica lag（複寫延遲）。

Multi-AZ：主要的掛了自動切過去

問題	答案
standby 平常在幹嘛	不接連線、不能讀寫，只收同步資料
什麼時候切	機器壞、AZ 出事、修補、手動重開並切換
切的時候發生什麼	DNS 名稱改指到 standby，它變成新的主要
要多久	官方說通常 60 到 120 秒
應用程式要改什麼	不用改連線字串，但要會斷線重連

換來的是已經寫成功的資料不因切換遺失；代價是多付一台不能用的機器錢。

standby 閒著，拿來分擔讀取？

× 大家通常會想

standby 閒著好浪費，報表連它就好。

這個版本的 standby 只收資料、不接連線，隨時準備接手。

✓ 實際上

要能讀的備援是另一種部署：**Multi-AZ DB cluster**。

一台寫、兩台可讀，只有 RDS for MySQL 與 PostgreSQL。

Read Replica：讀的人太多，分出去

promote（提升）：切斷複寫，把複本變成獨立可寫的資料庫；還沒抄到的那幾秒資料就沒了

問題	答案
平常在幹嘛	接報表、搜尋、首頁列表這類讀取
可以放哪	同 AZ、別的 AZ、甚至別的 Region
主要掛了會自動接手嗎	不會，要你手動 promote
會不會讀到舊資料	會，落後的那段就是 replica lag
應用程式要改什麼	要改，讀的查詢連複本自己的網址

剛寫完馬上要讀的查詢連主要；晚幾秒沒差的報表跟列表，才丟給複本。

考題看訊號詞：掛掉，還是讀太多

standby 同步刪表：Multi-AZ 防機器壞，不防手滑，手滑要靠時間點還原

題目裡看到	選
高可用、容錯、自動切換、防單一 AZ 出事	Multi-AZ
讀取量大、報表拖慢主要資料庫	Read Replica
跨 Region 的讀取、離海外使用者近	跨 Region 的 Read Replica
整個 Region 掛了也要救回來	跨 Region 的 Read Replica，出事時 promote

兩個可以同時開；Multi-AZ 救得了一個 AZ 出事，救不了整個 **Region**。

QUIZ 02 · CLF-C02

Amazon RDS 的哪一項功能可自動建立主要資料庫執行個體，並將資料同步複寫至另一個可用區 (Availability Zone) 中的執行個體？

A

讀取複本 (Read replicas)

B

藍綠部署 (Blue/green deployment)

C

Multi-AZ 部署 (Multi-AZ deployment)

D

保留執行個體 (Reserved Instances)

想 30 秒，等一下翻下一頁對答案。

QUIZ 02 · CLF-C02 · ANSWER

Amazon RDS 的哪一項功能可自動建立主要資料庫執行個體，並將資料同步複寫至另一個可用區 (Availability Zone) 中的執行個體？

A 讀取複本 (Read replicas)

B 藍綠部署 (Blue/green deployment)

C Multi-AZ 部署 (Multi-AZ deployment)

D 保留執行個體 (Reserved Instances)

C。「同步複寫到另一個 AZ」是 Multi-AZ 的定義；讀取複本是非同步，這一字之差就是這一課的重點。

AURORA

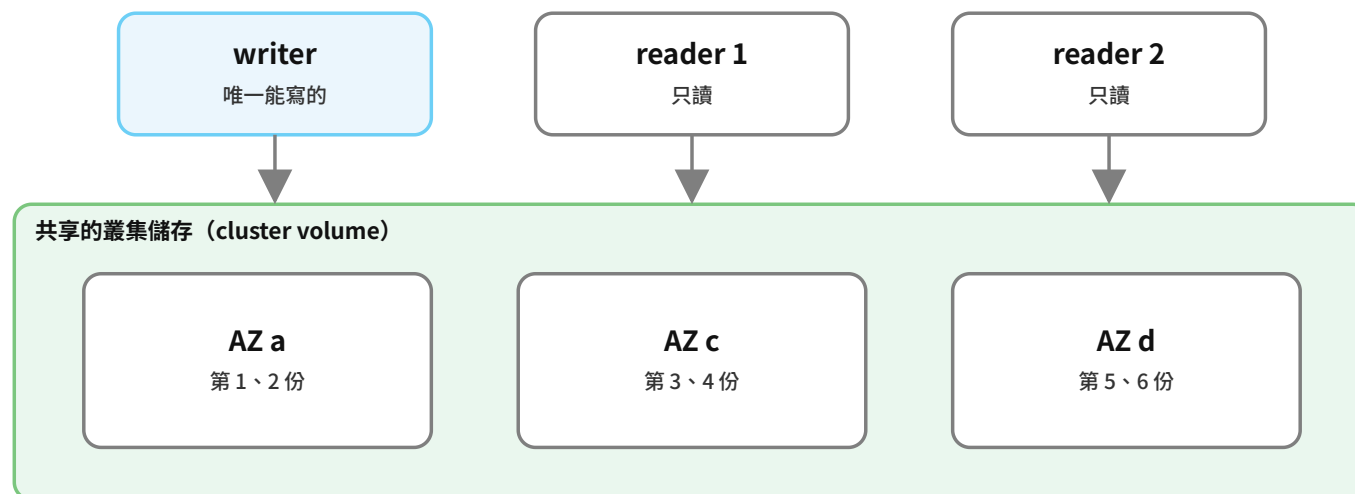
Aurora：運算跟儲存分開

- 一、一份共享儲存，跨三個 AZ 存六份
- 二、寫的連 **cluster endpoint**，讀的連 **reader endpoint**
- 三、**Serverless v2** 把挑機型變成設範圍，最小可到 0

不要每台各一份，大家共用一份

Aurora (AWS 重寫儲存層的資料庫，對外相容 MySQL 與 PostgreSQL) 把「算」跟「存」拆開

Aurora：運算跟儲存分開，儲存自己就跨三個 AZ 存六份



reader 看的是同一份儲存，不用再抄一遍資料；writer 掛了，挑一台 reader 升上來，通常 60 秒內

- 1 一台 writer、零到 15 台 reader，加上底下的叢集儲存，叫一個叢集
- 2 對機器來說是一份；底下 AWS 跨三個 AZ 存六份實體複本
- 3 儲存費只按實際用量算一次；機器費看你開幾台

RDS 的三個痛點，Aurora 各解一個

RDS 每台都有自己的磁碟，資料一份一份抄：同步就多等，非同步就落後

RDS 的痛點	Aurora 怎麼解
每個複本要自己抄一份資料	reader 讀共享儲存，落後通常遠小於 100 毫秒
想要備援就要多開一台 standby	儲存本身跨三個 AZ 存六份
切換要一兩分鐘	挑一台 reader 升成 writer，通常 60 秒內

六份複本是資料的高可用，reader 是運算的高可用。

儲存存六份，一台 **writer** 就夠？

✗ 大家通常會想

儲存已經跨三個 AZ，只開一台 **writer** 就高可用。

資料不會掉；但 **writer** 掛了要等重建，官方說通常 10 分鐘內。

✓ 實際上

正式環境至少一台 **reader** 放在別的 AZ。

整個 AZ 掛掉又沒有別的 AZ 的 **reader**，要你手動去開一台。

應用程式只記兩個網址

不用記每台機器的網址：寫的連一個、讀的連一個；最多 15 台 reader，reader endpoint 自己幫你分

網址	指向誰	給誰用
cluster endpoint （叢集端點）	永遠指向目前的 writer	寫入的程式；切換後自動改指
reader endpoint （讀取端點）	在所有 reader 之間分散	報表、列表這類只讀的查詢

既讀又寫的程式全連 **cluster endpoint** 也行，只是讀取沒分擔出去。

Serverless v2：挑機型變成設範圍

ACU (Aurora capacity unit)：約 2 GiB 記憶體加對應的 CPU，以 0.5 ACU 為單位增減；台數還是你自己加

功能	白話	什麼時候用
Serverless v2	設 ACU 範圍，每台在範圍內變強變弱	流量忽高忽低、很少用
縮到零	最小設 0，一段時間沒連線就暫停	開發環境、專題、內部工具
Global Database	主要 Region 寫，其他 Region 唯讀	海外讀得快、Region 掛了要切

從暫停醒來通常要 15 秒左右，超過 24 小時可能要 30 秒以上：[連線逾時不要設太短](#)。

Aurora 比較貴，值得時才用

講師補充，官方文件沒寫價差：同樣大小的機器，Aurora 通常比 RDS MySQL 貴一些

1 資料庫小、流量穩、不需要十幾台 reader

普通 RDS 加 Multi-AZ 就夠了。

2 讀取量大、要快速切換

共享儲存跟 reader 升級的好處用得到。

3 流量起伏大，想用 Serverless

閒著時縮小甚至暫停。

價格會變，去 Aurora Pricing 用你自己的數字算。

QUIZ 03 · CLF-C02

哪一個 **AWS** 服務是與 **MySQL** 和 **PostgreSQL** 相容的關聯式資料庫？

A

Amazon Redshift

B

Amazon DynamoDB

C

Amazon Aurora

D

Amazon Neptune

想 30 秒，等一下翻下一頁對答案。

QUIZ 03 · CLF-C02 · ANSWER

哪一個 **AWS** 服務是與 **MySQL** 和 **PostgreSQL** 相容的關聯式資料庫？

A

Amazon Redshift

B

Amazon DynamoDB

C

Amazon Aurora

D

Amazon Neptune

C。同時提供 MySQL 相容版與 PostgreSQL 相容版的關聯式資料庫是 Aurora；注意建叢集時要二選一，一個叢集只講其中一種語言，不是同一個資料庫兩種程式都能連。Neptune 是圖形資料庫。

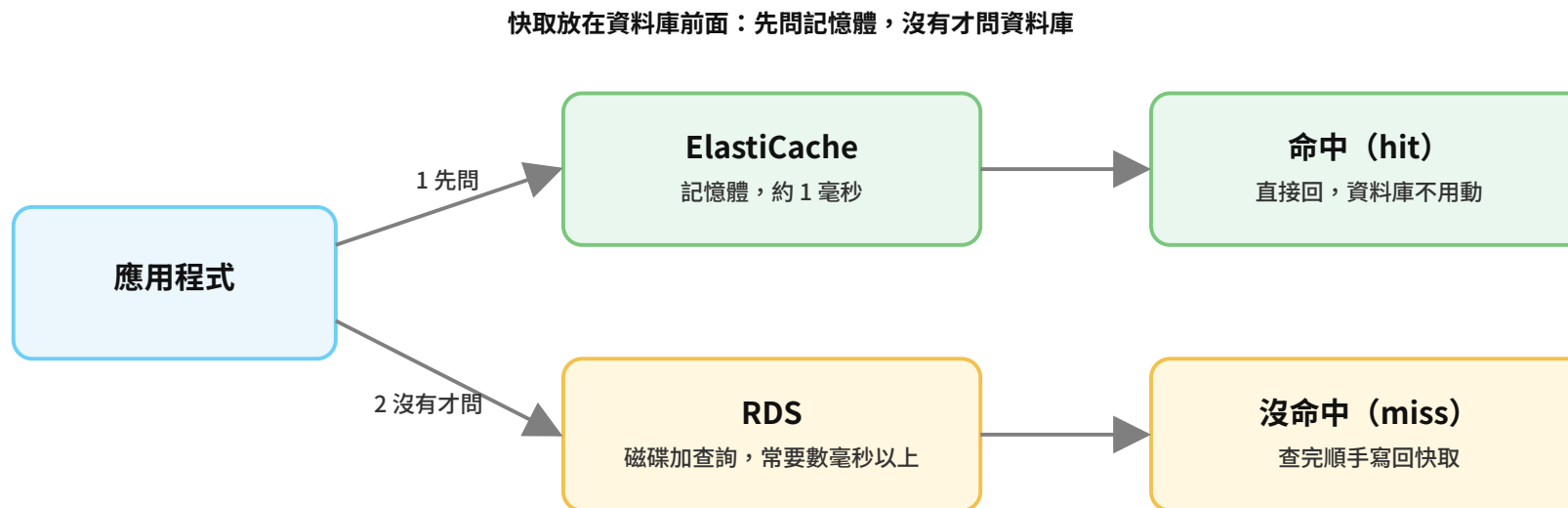
CACHE

快取：記下算好的答案

- 一、先問記憶體，沒有才問資料庫
- 二、資料庫是正本，快取是會消失的抄本
- 三、策略只答兩件事：什麼時候抄、能舊多久

先問記憶體，沒有才問資料庫

熱門商品一秒被問一千次、一小時才變一次；多開 reader 只是讓每台都重算同一個答案



數字是量級示意，實際看資料大小與查詢複雜度

ElastiCache：AWS 代管的記憶體快取服務；修補、備援、換壞節點是 AWS 的事

- 1 命中 (cache hit)：快取裡有，直接回，資料庫不用動
- 2 沒命中 (cache miss)：去問資料庫，查完順手寫回快取
- 3 命中率：一百次請求裡有幾次命中，快取有沒有用就看它

Valkey：Redis 改授權後的開源分支

鍵值（key-value）：一個名字對一個值，像程式裡的 dict；Redis 是最流行的記憶體鍵值資料庫

時間	發生什麼事
很多年	Redis 是開源的，雲端廠商拿來代管
2024 年 3 月	Redis 新版改成非開源授權（講師補充）
2024 年 3 月起	Linux 基金會維護的 Valkey 啟動，指令相容
2024 年 10 月	AWS 推出 ElastiCache for Valkey

官方說 Valkey 可以直接替換 Redis OSS，大多數程式換個端點就能用。

同樣的指令，便宜兩三成

官方定價頁的數字，都是 ElastiCache for Valkey 跟其他引擎比

1 Serverless 便宜 33%

不挑節點，用多少付多少。

2 自己挑節點便宜 20%

節點 (node)：固定記憶體大小的快取主機，有機型可挑。

3 Serverless 最低計費 100 MB

最低計費容量從 1 GB 降下來。

講師判斷：新專案直接選 Valkey。

部署跟引擎，是兩個獨立選擇

引擎官方有 Valkey、Memcached、Redis OSS 三種，這門課只講前兩種；引擎跟部署方式可以任意搭

選項	白話	什麼時候選
部署：Serverless	不挑節點，用多少付多少	專題、流量不好預測
部署：自己挑節點	挑機型跟台數，可以買預留	流量穩、要壓最低單價
引擎：Memcached	單純的大雜湊表，自挑節點版沒複寫	純快取、不需要資料結構

Valkey 能把清單當清單存：單獨取第 3 筆、尾巴加一筆，也能開複寫跟備份。

放進快取，就不會掉了？

× 大家通常會想

購物車只存在快取裡，反正記憶體很快。

節點重開、記憶體滿被擠掉、TTL 到期，都會不見。

✓ 實際上

資料庫是正本，快取只是抄本。

Valkey 9.0 以上的節點叢集可開 durability，只救節點壞掉。

QUIZ 04

首頁「熱門商品」每秒被問上千次、每小時才變一次，最划算的做法？

A

再加五台 Read Replica

B

在資料庫前面放 ElastiCache，把清單快取起來

C

把資料庫換成最大機型

D

改用 Multi-AZ

想 30 秒，等一下翻下一頁對答案。

QUIZ 04 · ANSWER

首頁「熱門商品」每秒被問上千次、每小時才變一次，最划算的做法？

A 再加五台 Read Replica

B 在資料庫前面放 ElastiCache，把清單快取起來

C 把資料庫換成最大機型

D 改用 Multi-AZ

B。答案很少變、被問很多次，是快取最典型的場景。**A** 有用，但每台 reader 都在重算同一個答案。

先算一筆帳：命中率決定一切

每秒總共 1000 次；快取 1 毫秒、資料庫 10 毫秒，沒命中那次先問快取再問資料庫，共 11 毫秒

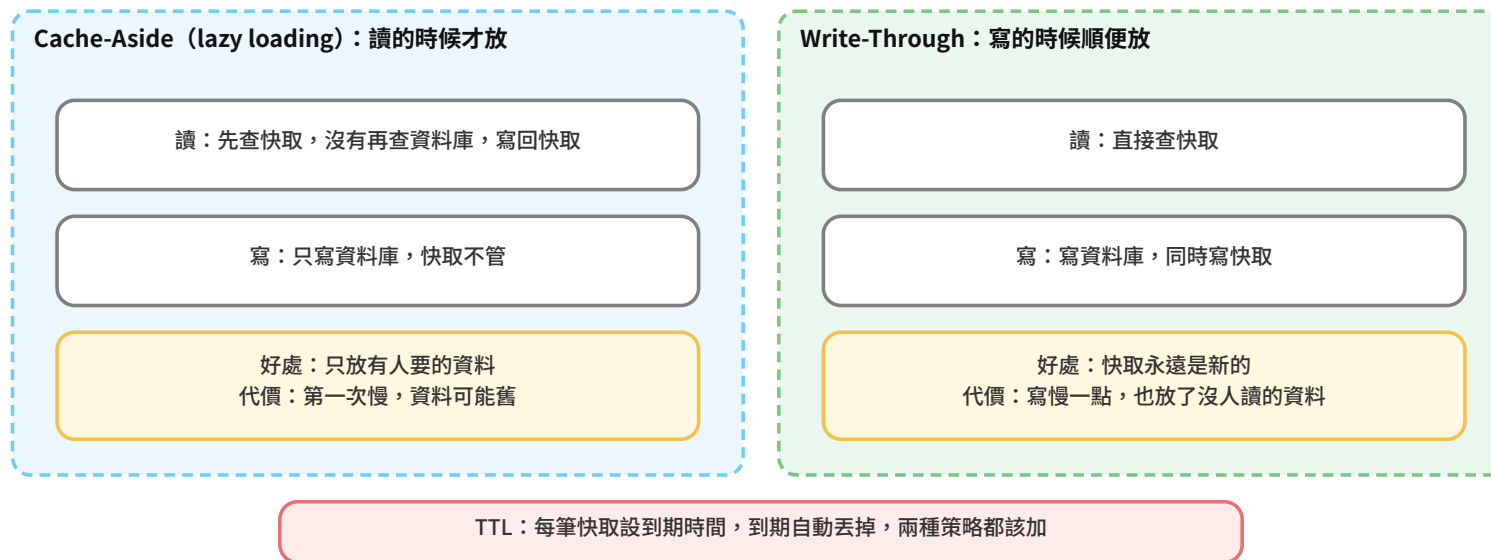
命中率	資料庫每秒要接幾次	平均延遲
0%（沒快取）	1000	10 毫秒
50%	500	$0.5 \times 1 + 0.5 \times 11 = 6$ 毫秒
90%	100	$0.9 \times 1 + 0.1 \times 11 = 2$ 毫秒

命中率從 50% 拉到 90%，資料庫壓力少了**五倍**；數字是示意，算法是真的。

用到才抄，還是一改就抄

Cache-Aside（旁路快取，又叫 lazy loading）讀到才放；Write-Through（寫穿）寫完資料庫，程式自己再寫快取

兩種寫快取的時機，加上一個保險 TTL



資料庫不會自動通知快取，Write-Through 的那次寫快取是你的程式做的

- 1 左邊 Cache-Aside：只放有人要的，第一次慢、資料可能舊
- 2 右邊 Write-Through：快取永遠是新的，寫慢一點
- 3 底下紅框：兩種都該加 TTL，到期自動丟掉

兩種時機各有代價，可以搭配

搭配的樣子：讀價格走 Cache-Aside，後台改價走 Write-Through，兩邊都掛 TTL

面向	Cache-Aside	Write-Through
快取裡放什麼	只有被讀過的	所有被寫過的，包括沒人讀的
資料會不會舊	會，資料庫改了快取不知道	不會，每次寫都更新
哪裡比較慢	沒命中那次讀，三趟	每次寫，兩趟
快取節點換新的	沒事，慢慢會補回來	下次寫入前資料都不在快取

講師經驗：起點是 Cache-Aside 加 TTL，寫入頻繁又要求新鮮的再補 Write-Through。

TTL：業務決定抄本能舊多久

TTL (time to live，存活時間)：從放進快取起算，時間到了這筆就當作不存在

TTL 設多長	會發生什麼
太短：資料其實幾小時才變	一直過期、一直回資料庫，命中率低
太長：又不主動刪快取	資料庫早改了，使用者還看到舊的
剛好	價格幾分鐘、熱門清單一小時、設定值一天

講師補充：一大批同一秒過期會打爆資料庫，TTL 往下抖一點隨機秒數錯開。

session 搬出機器，機器才能隨便砍

session：登入後的狀態，例如登入身分、購物車；瀏覽器用 cookie 自動帶著 session id

session 存在哪	自動擴展砍掉一台機器時
機器的記憶體裡	那台上的使用者全部被登出，購物車清空
ElastiCache	新機器拿同一個 session id 去查，照樣找到

session 外部化是機器變成無狀態的關鍵一步；放 Valkey 要開複寫跟備份，購物車最後仍要落到資料庫。

QUIZ 05

用 **Cache-Aside**，後台改了商品價格，前台還顯示舊價格十分鐘，最可能的原因？

- ☐ A 快取節點壞了
- ☐ B Cache-Aside 寫資料庫時不更新快取，舊的那筆要等 TTL 到期
- ☐ C 資料庫沒寫進去
- ☐ D 命中率太低

想 30 秒，等一下翻下一頁對答案。

QUIZ 05 · ANSWER

用 **Cache-Aside**，後台改了商品價格，前台還顯示舊價格十分鐘，最可能的原因？

- ☐ A 快取節點壞了
- ☒ B **Cache-Aside** 寫資料庫時不更新快取，舊的那筆要等 TTL 到期
- ☐ C 資料庫沒寫進去
- ☐ D 命中率太低

B。解法有三種：TTL 設短一點；改價時順便更新那筆快取（Write-Through）；或改價時直接刪掉那筆快取，讓下一次讀取重新去資料庫抓。

資料庫懂意思，切換要實測

- 一、**PostgreSQL** 裝 **pgvector** 就能用意思找文件
- 二、語意快取命中錯，比沒命中更危險
- 三、**Multi-AZ** 斷幾秒，自己量一次才算數

AI 應用要找的是意思接近

「買了之後多久內可以退」對上「商品到貨後七日內可申請退款」：一個字都沒對上，問的是同一件事

名詞	白話
embedding （嵌入向量）	把文字轉成一串數字，意思接近，數字也接近
向量檢索（ vector search ）	拿問題的那串數字，去找最接近的幾串
RAG （ Retrieval-Augmented Generation ）	先找相關文件，塞進提示詞，再交給模型回答

SQL 的模糊比對比的是字；向量檢索就是用意思找東西。

PostgreSQL 裝一個擴充就能做

pgvector：讓 PostgreSQL 能存向量、依距離排序的開源擴充；RDS 與 Aurora PostgreSQL 的擴充清單都有它

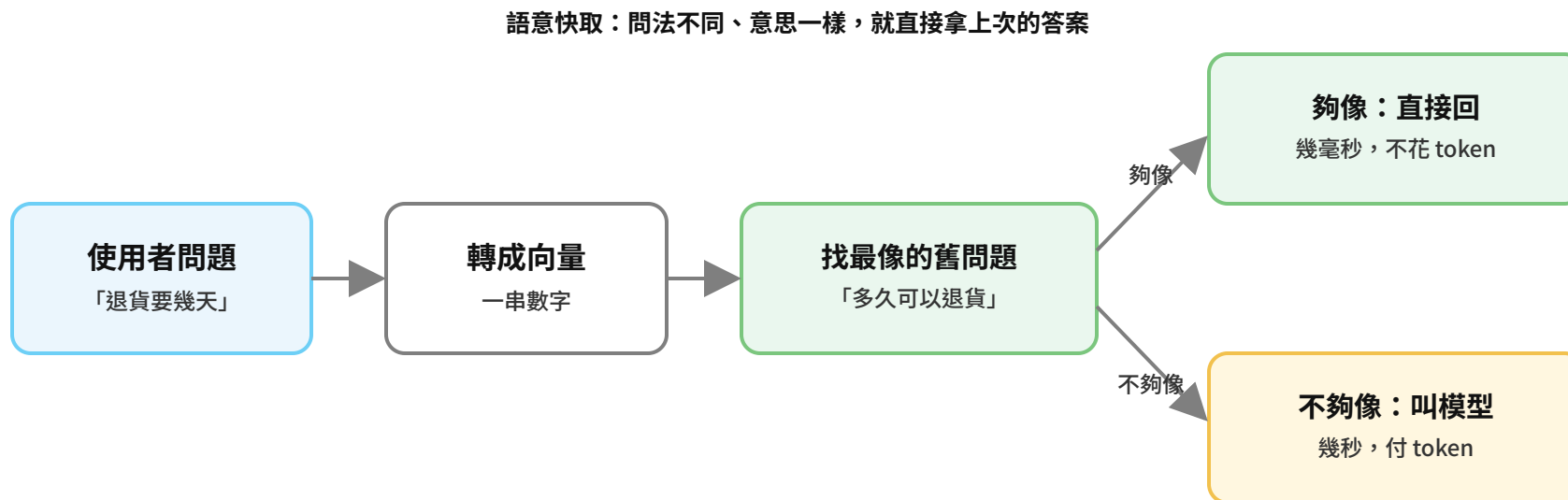
```
pgvector.sql

CREATE EXTENSION vector;
CREATE TABLE docs (id bigserial PRIMARY KEY,
  content text, embedding vector(1024));
-- 選用：建索引變成近似搜尋，資料多時通常快很多
CREATE INDEX ON docs USING hnsu (embedding vector_cosine_ops);
INSERT INTO docs (content, embedding)
  VALUES ('商品到貨後七日內可申請退款', '[1024 個數字]');
-- <=> 是餘弦距離，越小越像
SELECT content FROM docs
  ORDER BY embedding <=> '[問題轉出的 1024 個數字]' LIMIT 3;
```

- 1 數字由 embedding 模型算出，文件跟問題要用同一個模型
- 2 vector(1024) 的 1024 是那個模型輸出幾個數字
- 3 專題規模先用已經有的 PostgreSQL，不多開一個服務

換個說法，也能命中

語意快取 (semantic cache)：存「問題的向量對答案」，新問題只要意思夠接近就回舊答案



模型的答案寫回快取，下次同樣意思的問題就命中；多像才算夠像，是一個要調的門檻

LLM（大型語言模型）按 token（計費的最小文字單位）收錢；課綱量級：可省三到五成 token 成本

- 1 問題先轉成向量，去快取裡找最像的舊問題
- 2 夠像就回上次的答案：幾毫秒，不花 token
- 3 不夠像才叫模型，答案寫回快取，下次就命中

命中錯的，比沒命中更危險

「退貨要幾天」命中「換貨要幾天」，就是門檻太鬆

要調的東西	調不好會怎樣
多像才算命中（相似度門檻）	太鬆答錯；太緊幾乎不命中，白做
TTL	退貨政策改了，快取還在回舊答案
哪些問題不能快取	「我的訂單到哪了」：A 的答案給 B 就是外洩

講師規則：門檻寧可緊；跟使用者本人有關的問題，不進共用的語意快取。

門檻怎麼挑：從緊往鬆一格一格放

門檻是餘弦距離（ $\leq$ 算出來，0 到 2，越小越像）的上限：距離小於多少算同一個問題

1 湊 20 對該命中、20 對不該命中

不該命中的要挑長得像、但不是同一個問題的句子。

2 從 0.05 開始，每次放寬 0.05

每放一格，看該命中的命中幾對、不該命中的有沒有誤中。

3 出現第一對誤中，就退回上一格

那格的命中率就是收益；太低代表不適合語意快取。

agent 的記憶：看掉了要不要緊

agent（代理程式）：讓模型自己決定步驟、呼叫工具、多輪把任務做完的程式

放哪	適合
ElastiCache（Valkey）	這一輪對話的短期記憶，掉了頂多重講一次
RDS 或 Aurora	要查、要統計、要跟使用者資料 JOIN 的長期紀錄
DynamoDB、AgentCore 的記憶功能	後面的單元再比較

文件向量重算要花錢花時間，掉了會痛，放資料庫；對話記憶掉了不痛，放快取。

“

資料庫學會了「懂意思」，
快取也就跟著學會了。

—— 一句話帶走 ——

QUIZ 06

語意快取上線後，「我的訂單到哪了」會回別人的訂單狀態，根本原因？

A

TTL 太長

B

跟個人有關的問題被放進了共用的語意快取

C

embedding 模型太小

D

命中率太低

想 30 秒，等一下翻下一頁對答案。

QUIZ 06 · ANSWER

語意快取上線後，「我的訂單到哪了」會回別人的訂單狀態，根本原因？

A TTL 太長

B 跟個人有關的問題被放進了共用的語意快取

C embedding 模型太小

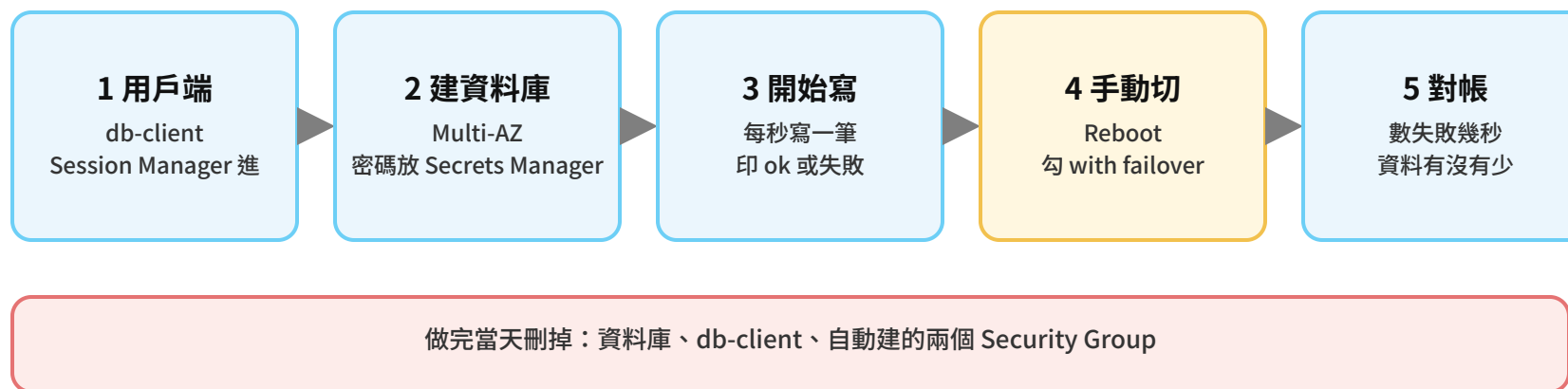
D 命中率太低

B。這類問題意思一模一樣、答案因人而異，一律不能進共用快取。

實作：手動切一次，記下斷了幾秒

一台小機器每秒寫一筆、印 ok 或失敗；你按下切換，記下失敗那段，最後查資料有沒有少

故障切換實測：一邊一直寫，一邊手動切，記下斷了幾秒



- 1 上排五格是必做的主線，零程式碼，只貼一段迴圈
- 2 第四格上色：手動切換是這堂課的主角
- 3 下排紅框是清理清單，做完當天刪掉

三層：必做七步就及格

選作，時間不夠改講師示範；零程式碼，只複製貼上一段迴圈

層	做什麼	配分
必做	管理者權限登入、區域切 Tokyo，照七步做完並對帳	70%
進階	拿官方事件時間對帳，看主要的 AZ 換了沒	20%
加分	不勾 failover 再重開一次，當對照組	10%

Multi-AZ 不在免費方案內：做完當天刪掉，整份實作花費以美分計。

必做：七步 (1/2)

步	做什麼	做完應該看到
1 用戶端	EC2 → Launch instance：名稱 db-client、Amazon Linux 2023、t3.micro、Key pair 選 Proceed without a key pair；Network settings 的網路用預設 VPC、Auto-assign public IP 保持 Enable（Session Manager 要能出網），選 Create security group 並取消勾選 Allow SSH traffic；Advanced details 的 IAM instance profile 選 ec2-ssm-role（沒有就照第 6 單元實作第 1 步建一個），User data 貼 <code>#!/bin/bash</code> 跟 <code>dnf install -y mariadb105</code> 兩行	兩三分鐘後 Status check 2/2 passed
2 建資料庫	RDS → Create database：Standard create、MySQL、Templates 選 Dev/Test、Availability 選 Multi-AZ DB instance deployment (2 instances)、識別名 lab-db、Master username admin、Credentials management 選 Managed in AWS Secrets Manager、機型選 Burstable 裡最小的 (db.t4g.micro)、Storage gp3 20 GiB	還不要按 Create
3 接上用戶端	同一頁 Connectivity 選 Connect to an EC2 compute resource，挑 db-client；Additional configuration 的 Initial database name 填 labdb；按 Create	狀態 Creating；講師實測要等 10 到 20 分鐘變 Available，Multi-AZ 欄是 Yes
4 拿連線資訊	lab-db → Connectivity & security 複製 Endpoint，順手記下同一頁顯示的 Availability Zone（進階要拿來比）；Configuration 分頁點 Master credentials ARN 進 Secrets Manager → Retrieve secret value，複製 password	一個 .rds.amazonaws.com 結尾的網址，一串密碼，一個像 ap-northeast-1a 的 AZ 名稱

必做：七步 (2/2)

步	做什麼	做完應該看到
5 開始寫	EC2 勾 db-client → Connect → Session Manager；先打 bash，再貼下面三行（網址跟密碼換成你的）	每秒一行 時間 ok
6 手動切	迴圈開著，另開一個分頁到 RDS → lab-db → Actions → Reboot → 勾 Reboot with failover → 按 Reboot	迴圈出現一段「失敗」，然後恢復 ok；恢復後再跑半分鐘就按 Ctrl+C
7 對帳	估中斷秒數：恢復後第一行 ok 的時間，減掉失敗前最後一行 ok 的時間（量法本身有誤差：每輪含 1 秒睡眠，連不上時最多再等 5 秒，估出來可能多幾秒）；再跑下面的查詢	COUNT 不小於迴圈最後印的成功次數；查詢結果按時間倒排，往下找到相鄰兩筆時間突然差幾十秒的地方，那就是失敗那段留下的「空洞」

第 5 步與第 7 步：複製貼上就好

在 Session Manager 先打 bash 再貼；網址跟密碼換成第 4 步拿到的

```
bash

export MYSQL_HOST=你的endpoint MYSQL_PWD='你的密碼'
mysql -u admin labdb -e "CREATE TABLE beat (t DATETIME(3))"
n=0; while true; do if timeout 5 mysql -u admin labdb \
  --connect-timeout=2 -e "INSERT INTO beat VALUES (NOW(3))" 2>/dev/null; \
  then n=$((n+1)); echo "$(date +%T) ok 第 $n 筆"; \
  else echo "$(date +%T) 失敗"; fi; sleep 1; done

# 第 7 步對帳
mysql -u admin labdb -e "SELECT COUNT(*) FROM beat;
  SELECT t FROM beat ORDER BY t DESC LIMIT 200"
```

- 1 每輪重新連線、重新解析網址，切換後自己接上
- 2 密碼放環境變數只為方便，正式程式跟 Secrets Manager 拿
- 3 官方提醒：正式環境先停寫入再切，這裡刻意邊寫邊切

對帳看兩件事：秒數跟筆數

中斷秒數用時間相減，不數失敗幾行：每輪含 1 秒睡眠，連不上時最多再等 5 秒

1 中斷秒數：恢復後第一行 ok 減失敗前最後一行 ok

量法本身有誤差，可能多估幾秒。

2 COUNT 應該不小於最後的成功次數

多出來的通常是寫好了、回應在切換中弄丟的那幾筆。

3 官方提醒勾 failover 的重開有掉資料的可能

少了就如實記下來，這一比就是實驗的重點。

進階與加分

層	做什麼	看到什麼算成功
進階	lab-db → Logs & events → Recent events，找切換開始與完成那兩筆事件的時間（講師實測看到的訊息大意是 Multi-AZ instance failover started 與 completed，字樣以主控台顯示為準）；再到 Connectivity & security 的 Availability Zone，跟第 4 步記下的 AZ 比，看主要所在的 AZ 是不是換了（切換後顯示可能晚幾分鐘才更新）	寫一句：官方事件記的秒數跟你迴圈記到的失敗秒數差多少、為什麼（提示：切換完成後，網址改指新的機器還要一點時間）
加分	先跑 <code>mysql -u admin labdb -e "TRUNCATE TABLE beat"</code> 清空資料表，迴圈重開，這次 Reboot 不勾 with failover，一樣估中斷秒數並對帳	兩次的秒數比一比，寫一句哪個比較久、AZ 有沒有換

卡住了：先翻提示卡

卡在哪	怎麼辦
第 2 步找不到 Multi-AZ	Templates 選到 Free tier，改選 Dev/Test
第 3 步清單裡沒有 db-client	機器還沒 running，或不在同一個區域
第 5 步 mysql: command not found	<code>sudo dnf install -y mariadb105</code> 補裝
第 5 步一直印失敗	單跑一次 <code>SELECT 1</code> 看錯誤：密碼或連線
第 6 步沒有 with failover 可勾	資料庫不是 Multi-AZ，回第 2 步

Access denied 多半是密碼沒複製完整；**Can't connect** 多半是第 3 步沒選 db-client 或還沒 Available。

清理

順序	做什麼
1	RDS → lab-db → Actions → Delete：取消勾選 Create final snapshot 跟 Retain automated backups，勾確認，輸入 delete me
2	EC2 → db-client → Terminate
3	兩者都刪完後，到 Security Groups 找第 3 步自動建的兩個（名稱像 ec2-rds-1、rds-ec2-1）；它們的規則互相指著對方；講師實測：規則還指著對方時 AWS 不讓你刪群組，要先各自把指向對方的那條規則刪掉，兩個群組才刪得掉
4	Secrets Manager 裡 RDS 建的那個 secret，官方說會跟著資料庫一起刪掉，看一眼確認（Password management with Amazon RDS and AWS Secrets Manager，2026-09-24 查閱）

實作繳交：兩張截圖、一句對帳

個人完成；只交必做就及格，時間不夠看講師示範的也可以繳一句觀察

- 1 迴圈失敗那段的截圖
- 2 第 7 步查詢結果的截圖：空洞前後的時間與 COUNT
- 3 一句話：斷了幾秒、COUNT 跟最後成功次數比起來如何
- 4 做了進階附事件時間與解釋；做了加分附兩次秒數比較

驗收 必做 70%（兩張截圖各 25%、那一句 20%），進階 20%，加分 10%。

QUIZ 07 · CLF-C02

一家公司希望能安全地儲存 **Amazon RDS** 資料庫的憑證，並且能定期自動輪替使用者密碼。哪一項 **AWS** 服務或功能可以滿足這些需求？

A

Amazon S3

B

AWS Systems Manager Parameter Store

C

AWS Secrets Manager

D

AWS CloudTrail

想 30 秒，等一下翻下一頁對答案。

QUIZ 07 · CLF-C02 · ANSWER

一家公司希望能安全地儲存 **Amazon RDS** 資料庫的憑證，並且能定期自動輪替使用者密碼。哪一項 **AWS** 服務或功能可以滿足這些需求？

A

Amazon S3

B

AWS Systems Manager Parameter Store

C

AWS Secrets Manager

D

AWS CloudTrail

C。存 RDS 憑證又要自動輪替，選 Secrets Manager；Parameter Store 能存加密參數，但沒有內建的資料庫密碼輪替。

回家作業：拿期末專題來做，一律個人完成

每一份接著上一份做；實作的繳交看實作那一頁

自己養還是交給 RDS

三列的表：引擎、RDS 或 EC2 自建、理由。

Multi-AZ 與 Read Replica

兩行：各要不要開，理由用「掛掉」或「讀太多」。

RDS 還是 Aurora

三行：選哪個、用到哪個功能、一句成本理由。

哪個查詢值得快取

三個讀取查詢：多久變一次、一秒問幾次，挑一個。

替那個查詢定策略

三行：哪種策略、TTL 多少、最多看到多久的舊資料。

快取與向量放哪

三列的表：文件向量、語意快取、對話記憶各放哪。

權限與資料表

TAKEAWAY

- 01** 受管買的是機器層半夜不用起床；**權限與資料表**永遠是你的。
- 02** Multi-AZ 是保險、Read Replica 是分身；Aurora 讓大家**共用一個倉庫**。
- 03** 快取讓資料庫**少做事**；策略只答什麼時候抄、抄本能舊多久。
- 04** 語意快取命中錯比沒命中危險；高可用要**量過才算數**。

Reference

- **RDS User Guide : What is Amazon RDS 、 Introduction to backups** [受管與備份](#)
- **RDS User Guide : Restoring a DB instance to a specified time** [時間點還原](#)
- **RDS User Guide : Multi-AZ DB instance deployments 、 Working with read replicas** [兩種多一台](#)
- **Aurora User Guide : Amazon Aurora storage 、 High availability for Aurora** [共享儲存與切換](#)
- **Aurora User Guide : How Aurora serverless works 、 Using Aurora Global Database** [Serverless v2 與全球資料庫](#)
- **ElastiCache : Caching strategies 、 Comparing Valkey, Memcached, and Redis OSS** [快取策略與引擎](#)
- **Extension versions for RDS for PostgreSQL ; pgvector** [向量檢索](#)
- **RDS User Guide : Failing over a Multi-AZ DB instance 、 Rebooting a DB instance** [故障切換實作](#)